

AUDITORÍA COMPLETA — bitget-ai-trading-bot

Por: El Mayor Experto en Crypto Trading, Matemáticas Financieras y Algoritmos

CONTEXTO Y OBJETIVO

Este documento es una auditoría forense y estratégica del repositorio bitget-ai-trading-bot. El bot opera con futuros perpetuos en Bitget (USDT-M) sobre cuentas pequeñas (~40 USDT). Se analiza la arquitectura, lógica de señales, gestión de riesgo, matemáticas, performance, seguridad y potencial de rentabilidad. El objetivo es identificar bugs, debilidades, ineficiencias y oportunidades de mejora para maximizar el edge real del sistema.

I. RESUMEN EJECUTIVO DE HALLAZGOS

Nivel | Cantidad | Descripción breve

--- | --- | ---

CRÍTICO | 5 | Bugs o lógicas que destruyen directamente el PnL

ALTO | 8 | Ineficiencias graves o riesgos sistémicos

MEDIO | 9 | Mejoras de rendimiento y edge

BAJO/MEJORA | 6 | Optimizaciones opcionales

II. ARQUITECTURA GENERAL

Tecnología: Python puro, REST polling, SQLite/PostgreSQL, deployado en Railway/Docker.

Flujo de un ciclo (cada 30s):

```
fetch_all(symbols) → detect_regime(BTC/ETH) → generate_signals(∀ symbols)
→ allocate() → [meta_model?] → risk_manager.validate()
→ execution_engine.execute() → position_manager.monitor()
→ labeler.label() → mfe_mae_tracker.update()
```

Archivos críticos del core:

- app/signal_engine.py — Motor de señales (scoring 0–100)
- app/risk_manager.py — Validación de riesgo y sizing
- app/market_data.py — Proveedor de datos OHLCV
- app/regime_detector.py — Detector de régimen BTC/ETH
- app/portfolio_allocator.py — Selección de señales
- app/execution_engine.py — Ejecución real/paper/dry
- app/paper_trader.py — Simulación paper trading
- app/config.py — Configuración centralizada (624 líneas)
- app/main.py — Bucle principal (1393 líneas)

III. HALLAZGOS CRÍTICOS

■ CRÍTICO-1: El R:R real no incluye fees+slippage en el cálculo de score

Archivo: app/signal_engine.py:147

```
risk_reward = abs(take_profit_1 - entry) / risk_per_unit # INCORRECTO
```

Problema: El ratio R:R con el que el bot decide si operar ignora las comisiones round-trip (~0.12%) y el slippage (~0.03%). Para posiciones con stop estrecho (0.6% min), esto sobreestima el R:R real en **10–25%**.

Impacto matemático real:

- Stop de 0.6%, TP1 = +0.96% → R:R declarado = 1.6
- Fees+slippage = ~0.15% cada lado → Coste real = 0.30%
- R:R neto real = $(0.96 - 0.15) / (0.6 + 0.15) = 0.81 / 0.75 = \mathbf{1.08}$ (no 1.6)
- El bot cree que gana 1.6:1 cuando en realidad gana 1.08:1

Corrección: Calcular R:R neto incluyendo costes:

```
cost_rate = rules.taker_fee_rate * 2 + 0.0003 # fees + slippage
net_profit_tp1 = abs(take_profit_1 - entry) - entry * cost_rate
net_risk = risk_per_unit + entry * cost_rate
risk_reward = net_profit_tp1 / net_risk
```

■ CRÍTICO-2: TP levels fijos en 1.6x y 2.4x — matemáticamente sub-óptimos

Archivo: app/signal_engine.py:145-146

```
take_profit_1 = entry + risk_per_unit * 1.6 # Hardcoded
take_profit_2 = entry + risk_per_unit * 2.4 # Hardcoded
```

Problema: Niveles fijos ignorantes del régimen, volatilidad y estructura de mercado. En mercados trending, TP1 a 1.6x es demasiado conservador (se cierra muy pronto). En mercados choppy, TP2 a 2.4x es inalcanzable (tiempo esperado demasiado largo).

Evidencia matemática: Si la estrategia tiene win_rate = 45% con R:R neto real de 1.08 (ver CRÍTICO-1), el Expectancy es:

$$E = 0.45 \times 1.08 - 0.55 \times 1 = 0.486 - 0.55 = -0.064 \text{ (EDGE NEGATIVO)}$$

Corrección: TP dinámico basado en ATR y régimen:

- TREND_UP/DOWN: TP1 = 2.0× risk, TP2 = 3.5× risk
- RANGE/CHOPPY: TP1 = 1.4× risk, TP2 = 1.9× risk (objetivos más realistas)
- BREAKOUT: TP1 = 2.5× risk, TP2 = 4.0× risk

■ CRÍTICO-3: Stop Loss demasiado mecánico — ATR sin estructura de mercado

Archivo: app/signal_engine.py:234-241

```
@staticmethod
def _calculate_stop(side, entry, atr, row):
    if side == "LONG":
        support = safe_float(row.get("support_recent"))
        structure_stop = support if support and support < entry else entry - atr * 1.4
        return min(entry - atr * 1.1, structure_stop)
    # SHORT análogo...
```

Problema: El stop toma el min() entre ATR×1.1 y soporte estructural. Esto significa que SIEMPRE usa el stop más CERCANO, ignorando cuál tiene más lógica de mercado. Un stop de ATR×1.1 puede estar dentro del "ruido" del mercado, causando falsas activaciones.

Segundo bug: structure_stop = support if support and support < entry else entry - atr * 1.4 — cuando no hay soporte, el fallback es ATR×1.4, pero luego hace min(ATR×1.1, ATR×1.4) → siempre elige ATR×1.1.

El fallback de 1.4x es inútil.

Impacto: Tasa de whipsaw (stops falsos antes de que el precio vuelva a la dirección correcta) artificialmente alta.

Corrección:

```
# Priorizar soporte estructural cuando existe y es coherente
if support and entry * 0.994 > support > entry * 0.975: # rango válido [0.6%-2.5%]
    return support - entry * 0.002 # buffer debajo del soporte
return entry - max(atr * 1.4, entry * config.min_stop_distance_pct)
```

■ CRÍTICO-4: Race condition en live trading — balance obsoleto entre validación y ejecución

Archivo: app/main.py:481-499

```
# Primero valida con balance_anterior
risk = risk_manager.validate_signal(signal, balance=balance, ...)

# Luego refresca balance para CADA señal individualmente
if config.can_send_real_orders:
    balance, available_balance, used_margin, balance_ok = \
        _refresh_live_account_balance(...)

# Después usa effective_balance (que puede diferir del que validó)
effective_balance = balance * 0.5 if news.reduce_risk else balance
```

Problema: La validación de riesgo se hace con balance del ciclo anterior. Si hay un cambio de balance (otra posición cerrada, funding rate, etc.) entre ciclos, el sizing calculado no corresponde al balance real al momento de ejecutar. En modo live, esto puede causar:

1. Posiciones sobredimensionadas si el balance bajó
2. Validaciones incorrectas de límites diarios/semanales
3. El effective_balance pasado al risk_manager es distinto al usado en el refresh

Corrección: Refrescar balance ANTES de la validación de riesgo, no después:

```
if config.can_send_real_orders:
    balance, available_balance, used_margin, balance_ok = _refresh_live_account_balance(...)
    if not balance_ok:
        continue
risk = risk_manager.validate_signal(signal, balance=balance, available_balance=available_balance, ..)
```

■ CRÍTICO-5: Circuit breaker ciega a magnitud — 3 micro-pérdidas = mismo bloqueo que 3 pérdidas grandes

Archivo: app/risk_manager.py:67-70

```
def set_consecutive_losses(self, count: int) -> None:
    self.consecutive_losses = count
    if count >= self.config.max_consecutive_losses:
        self.cooldown_until = datetime.now(timezone.utc) + timedelta(
            minutes=self.config.cooldown_after_losses_minutes)
```

Problema: 3 pérdidas de \$0.01 cada una activan el mismo cooldown de 3 horas que 3 pérdidas del 2.5% del balance. Además, el circuit breaker no distingue entre pérdidas en condiciones de mercado similares vs. distintas. En los mercados reales, series cortas de pérdidas pequeñas son estadísticamente normales (drawdown esperado).

Falta: No hay circuit breaker para pérdida acumulada % del balance (drawdown máximo). Solo hay circuito por número de pérdidas.

Corrección: Circuit breaker dual — por conteo Y por magnitud:

```
# Activar solo si pérdida acumulada supera umbral (ej: >3% del balance en las N pérdidas)
cumulative_loss = sum(recent_loss_amounts[-max_consecutive_losses:])
if count >= max_consecutive_losses and cumulative_loss > balance * 0.03:
    activate_cooldown(...)
```

IV. HALLAZGOS DE NIVEL ALTO

■ *ALTO-1: Datos de mercado descargados secuencialmente — latencia de 3–8 segundos*

Archivo: app/market_data.py:85–91

```
def fetch_all(self, symbols: list[str]) -> dict[str, MarketSnapshot]:
    snapshots = {}
    for symbol in symbols:
        # Secuencial: 10 símbolos × 300ms = 3 segundos
        snapshot = self.fetch_symbol(symbol)
```

Y dentro de `fetch_symbol`, los `timeframes` también son secuenciales:

```
for timeframe in dict.fromkeys(self.timeframes): # 5 timeframes × 200ms = 1 segundo por símbolo
    raw = self.client.get_candles(symbol, api_timeframe, limit=limit)
```

Impacto cuantificado:

- 10 símbolos × 5 timeframes × ~150ms/request = ~7.5 segundos de fetch
- En un ciclo de 30s, el 25% del tiempo es fetching
- Peor: los datos del símbolo #10 llegan 7 segundos después que el símbolo #1 → timestamps desincronizados
- Las señales se generan con datos "de distintos momentos"

Corrección: `ThreadPoolExecutor` para fetch paralelo:

```
from concurrent.futures import ThreadPoolExecutor, as_completed

def fetch_all(self, symbols):
    with ThreadPoolExecutor(max_workers=min(len(symbols), 5)) as executor:
        futures = {executor.submit(self.fetch_symbol, s): s for s in symbols}
        return {futures[f]: f.result() for f in as_completed(futures) if not f.result().error}
```

Esto reduce el fetch de 10 símbolos de ~7.5s a ~1.5s.

■ *ALTO-2: No hay WebSocket — polling REST cada 30s es ciego a movimientos rápidos*

Problema: El bot no tiene WebSocket. Usa REST polling con `SCAN_INTERVAL_SECONDS=30`. Un movimiento violento (ej: pump/dump en 10 segundos) puede:

1. Abrir posición en el peor momento (tras el movimiento)
2. Perder señales de alta calidad que ya "caducaron"
3. No detectar stops alcanzados en tiempo real (solo en el `paper_trader` el monitor está a 5s)

Impacto: En futuros perpetuos, los movimientos de $\pm 2\%$ en 30 segundos son frecuentes. El bot opera con información que puede tener hasta 30s de edad.

Corrección sugerida (mediano plazo): Implementar WebSocket para tickers en tiempo real (Bitget soporta `wss://ws.bitget.com`). El ciclo de 30s se mantiene para señales, pero el monitor de posiciones y los triggers de salida deben operar en tiempo real.

■ *ALTO-3: Régimen de mercado solo usa BTC/ETH — ignora divergencias de altcoins*

Archivo: app/regime_detector.py:41–55

```
btc = snapshots.get("BTCUSDT")
eth = snapshots.get("ETHUSDT")
# Todo el análisis se basa SOLO en BTC y ETH
eth_bias = trend_bias(eth.candles.get("15m")) if eth and "15m" in eth.candles else "neutral"
```

Problema: Si SOL, ADA o LINK tienen un movimiento específico de su ecosistema (hack, upgrade, partnership) que no mueve BTC, el bot lo tratará como "RANGE" o "CHOPPY" y no operará. O peor, si BTC baja 0.5% y SOL sube 3%, el bot bloquea LONGS por "RISK_OFF BTC".

Corrección: Régimen híbrido con "régimen per-símbolo" + "régimen global":

- global_regime = análisis BTC/ETH (actual)
- symbol_regime = análisis del símbolo específico (nuevo)
- Si symbol_regime es muy divergente del global_regime → penalizar señal pero no bloquear

■ **ALTO-4: Grupos de correlación estáticos — no refleja correlaciones dinámicas del mercado**

Archivo: app/portfolio_allocator.py:20-31

```
CORRELATION_GROUPS = {
    "BTCUSDT": "majors",
    "ETHUSDT": "majors",
    "SOLUSDT": "majors",
    "BNBUSDT": "majors",
    # ...
}
```

Problema: Las correlaciones cripto son altamente dinámicas. SOL puede tener correlación de 0.95 con BTC en mercados de riesgo, pero solo 0.3 durante eventos de su propio ecosistema. Tratar siempre a SOL como "major" correlacionado con BTC impide abrir oportunidades descorrelacionadas.

Corrección: Calcular correlación rolling de 24h entre cada par y BTC usando los datos de OHLCV que ya se tienen, y actualizar CORRELATION_GROUPS dinámicamente cada ciclo.

■ **ALTO-5: Trailing stop declarado pero nunca implementado en live**

Archivo: app/signal_engine.py:221-222

```
trailing_stop_enabled=score >= self.config.min_score_excellent,
trailing_stop_rule="ATR 1.2 tras TP1; mover SL a break-even",
```

Problema: El campo trailing_stop_enabled se calcula y se guarda, pero **nunca se actúa sobre él** en el execution_engine.py ni en el position_manager.py. El "trailing stop" es un comentario de texto, no lógica ejecutada.

Impacto: Las posiciones con score ≥85 podrían proteger ganancias automáticamente. Sin trailing stop, la posición lleva el riesgo completo hasta TP2 o SL, degradando el ratio riesgo-beneficio ajustado a tiempo.

Corrección: Implementar la lógica de mover SL a break-even cuando TP1 se alcanza (ya existe en paper_trader.py para paper — falta en ejecución live).

■ **ALTO-6: El stop en paper_trader usa comisión fija 0.06% — incorrecto para cuentas pequeñas**

Archivo: app/paper_trader.py:90

```
fees = signal.entry_price * signal.position_size * 0.0006 # 0.06% taker fee SOLO apertura
```

Problemas:

1. Solo calcula fees de apertura, NO de cierre
2. 0.06% es la tarifa estándar, pero sin BGB token pueden ser 0.08–0.1%
3. No incluye funding rate (relevante para posiciones >4 horas)

Corrección: Calcular fees round-trip completo:

```
round_trip_fees = notional * rules.taker_fee_rate * 2 # apertura + cierre
funding_estimate = notional * 0.0001 * estimated_hours / 8 # funding cada 8h
total_cost = round_trip_fees + funding_estimate
```

■ **ALTO-7:** `_max_positions_for_balance` *tiene bug de lógica duplicada*

Archivo: `app/risk_manager.py:549-554`

```
def _max_positions_for_balance(self, balance: float) -> int:
    if balance < 60:
        return self.config.small_account_max_open_positions
    if balance <= 60 and not self.config.allow_second_position_small_account: # ← DEAD CODE
        return self.config.small_account_max_open_positions
    return min(...)
```

Bug: La segunda condición `if balance <= 60` nunca se ejecuta porque la primera ya captura `balance < 60`, y `balance == 60` es un caso borde que llega a la segunda condición pero también está capturado. La segunda rama es código muerto.

Impacto: Menor (para cuentas exactamente en 60 USDT el comportamiento puede ser incorrecto).

■ **ALTO-8:** `fetch_symbol` *descarga 220 velas siempre — despilfarro de 80% de los datos*

Archivo: `app/market_data.py:59`

```
def fetch_symbol(self, symbol: str, limit: int = 220) -> MarketSnapshot:
```

Problema: Descarga 220 velas cada 30 segundos pero solo necesita los últimos 60 valores para los indicadores. Las velas más antiguas se recalculan con los mismos datos en cada ciclo. Esto supone:

- 4× más datos de los necesarios por request
- 4× más tiempo de parsing y cálculo de indicadores
- En 10 símbolos × 5 timeframes = 50 requests × 160 velas extra = 8000 rows parseados innecesariamente por ciclo

Corrección: Cache incremental — cachear las últimas N velas y solo pedir las nuevas desde el último timestamp conocido.

V. HALLAZGOS DE NIVEL MEDIO

■ **MEDIO-1:** *Score system puede ser "jugado" — confirmaciones fáciles suman más que las difíciles*

Archivo: `app/signal_engine.py:105-171`

El sistema de puntuación tiene pesos fijos con problemas de diseño:

Condición | Puntos | Problema

--- | --- | ---

`bias5==bias15==deseado` | +20 | En tendencia fuerte, siempre se cumple sin añadir información real

bias1h no contradice | +10 | "neutral" puntúa positivo — fácil de cumplir
EMAs a favor | +15 | En tendencia, siempre se cumple junto con bias (doble conteo)
MACD a favor | +12 | Correlacionado con bias — triple conteo
RSI sano | +10 | Rango muy amplio (45-72 para LONG) — casi siempre positivo

Problema: 4 de las 5 principales confirmaciones están altamente correlacionadas entre sí. Una tendencia bullish fuerte puede sumar +67 puntos de facto con señales que miden lo mismo de distintas formas. El sistema no penaliza suficientemente la divergencia entre indicadores del mismo tipo.

Corrección: Aplicar pesos de importancia basados en backtesting de señales individuales (MFE/MAE ya capturado puede usarse para calibrar). Agrupar indicadores correlacionados y puntuar el grupo en bloque.

■ **MEDIO-2:** `clamp(score, 0, 100)` *elimina información de sobreconfianza*

Archivo: `app/signal_engine.py:202`

```
score = int(clamp(score, 0, 100))
```

Problema: Un score de 130 (señal muy fuerte) se trata igual que un score de 100. La magnitud por encima de 100 es información valiosa para ordenar señales. El allocator entonces no puede distinguir entre una señal de 100 y una de 130.

Corrección: Permitir score sin clamping para uso interno del allocator, y solo clampear para display/logging:

```
signal.raw_score = score # Sin clamping  
signal.confidence_score = int(clamp(score, 0, 100)) # Para display
```

■ **MEDIO-3:** *News intel hace veto total sin modo gradual*

Archivo: `app/main.py:351-353`

```
if news.block_trading:  
    selected = [] # Cancela todo
```

Y en `app/main.py:399-400`:

```
if news.reduce_risk:  
    effective_balance = balance * 0.5 # Solo reduce sizing
```

Problema: El modo `block_trading` es todo-o-nada. No hay gradación: "bloquear solo longs", "reducir score mínimo a 90", "reducir tamaño al 25%". En alta volatilidad por noticias, algunos trades de muy alta calidad pueden seguir siendo válidos.

■ **MEDIO-4:** *El labeler de triple barrera usa `MAX_HOLDING_BARS=48` sin considerar timeframe*

Archivo: `app/config.py:218`

```
max_holding_bars: int = 48
```

Problema: 48 barras en 5m = 4 horas. Pero algunas estrategias (`TREND_FOLLOWING`) pueden necesitar más tiempo. Posiciones válidas que se resuelven a las 6h se etiquetan como "TIME" (label 0) cuando en realidad eran ganadores diferidos.

Impacto: Contamina el dataset de entrenamiento del meta-model con falsos negativos.

■ **MEDIO-5:** *Indicadores calculados de nuevo desde cero en cada ciclo — CPU wasteful*

Archivo: app/market_data.py:79

```
snapshot.candles[timeframe.lower()] = add_indicators(df)
```

Problema: Cada ciclo recalcula EMA21, EMA50, RSI, MACD, ATR, etc. para todas las 220 barras de cada símbolo, cuando solo la última barra es nueva. Con 10 símbolos × 5 timeframes × ~20 indicadores = 1000 recálculos por ciclo de variables que 99% ya se calcularon en el ciclo anterior.

Corrección: Indicadores incrementales — cachear los estados intermedios (EMA state, RSI wilder smoothing) y solo actualizar con la nueva barra.

■ MEDIO-6: No hay validación de sincronización de reloj con servidor Bitget

Problema: La firma HMAC incluye timestamp del sistema. Si el reloj del host diverge >5 segundos del servidor Bitget, todas las órdenes son rechazadas con error de autenticación. En Railway/Docker, esto puede ocurrir tras una hibernación o migración de container. No hay ningún chequeo ni sincronización NTP.

Corrección: Sincronizar timestamp con la API de Bitget al startup y detectar divergencia:

```
server_time = client.get_server_time()
local_time = now_ms()
drift_ms = abs(server_time - local_time)
if drift_ms > 4000:
    logger.error("Clock drift %dms - autenticación fallará", drift_ms)
    telegram.critical(f"Reloj desincronizado: {drift_ms}ms de deriva")
```

■ MEDIO-7: Funding rate solo considerado como bonus de score, no como coste

Archivo: app/signal_engine.py:161-165

```
if snapshot.funding_rate:
    funding_favorable = snapshot.funding_rate <= 0 if proposed_side == "LONG" else snapshot.funding_rate >= 0
    if funding_favorable:
        score += 5
```

Problema: El funding rate SOLO suma si es favorable, pero no resta si es desfavorable y alto. En algunos pares, el funding rate puede ser >0.1% cada 8 horas (3.65% mensual). Para una posición mantenida 8h+, el funding rate puede devorar todo el TP1.

Corrección: Incluir funding como coste explícito en el cálculo de R:R:

```
# Estimar horas de holdeo: max_holding_bars * timeframe_minutes / 60
estimated_hold_hours = config.max_holding_bars * 5 / 60 # 48 barras * 5m = 4h
funding_cost = abs(snapshot.funding_rate) * notional * (estimated_hold_hours / 8)
# Ajustar TP para cubrir funding:
adjusted_tp1_distance = abs(take_profit_1 - entry) - funding_cost
```

■ MEDIO-8: Volume relative calculado sin contexto de sesión de mercado

Problema: volume_relative compara volumen actual vs. media. Pero el volumen de crypto tiene patrones intradiarios fuertes (bajo volumen a las 4am UTC, alto a las 14-20h UTC). Un volumen "1.8x la media" a las 4am puede ser menor en absoluto que "0.9x" a las 14h.

Corrección: Normalizar volumen por hora del día (basado en histórico de las últimas N semanas para esa hora).

■ MEDIO-9: El MetaModel está en observe_only — nunca filtra trades

Archivo: app/config.py:217

```
meta_model_mode: str = "observe_only"
```

Problema: El meta-model (ML sobre señales históricas) está implementado pero desactivado para filtrar. Solo observa. Acumula datos pero no se usa para mejorar decisiones. Cuando se cambia a filter, requiere 300 samples labeled + 50 positivos + 50 negativos, lo que puede tardar semanas en lograrse con la frecuencia de operaciones actual.

Oportunidad: Activar meta_model_mode=filter con umbrales conservadores (meta_min_probability=0.60) una vez acumulados suficientes datos.

VI. HALLAZGOS DE NIVEL BAJO / MEJORAS DE RENDIMIENTO

■ **BAJO-1:** *Score mínimo de 72 puede ser demasiado bajo para edge real*

Con un score de 72 posible con solo 3 confirmaciones correlacionadas (ver MEDIO-1), el mínimo efectivo real puede ser en torno a 80 para tener edge estadístico. Recomendación: elevar MIN_SCORE_TO_TRADE a 78–80 tras analizar win_rate por rango de score en los datos de MFE/MAE.

■ **BAJO-2:** *No hay índices DB en queries calientes*

signal_observations y signal_labels son las tablas más consultadas pero no tienen índices en columnas de filtrado frecuente (timestamp, symbol, labeled, score). Con miles de registros, las queries se vuelven lentas.

Corrección: Añadir índices en database.py:

```
CREATE INDEX IF NOT EXISTS idx_obs_labeled ON signal_observations(labeled, timestamp);
CREATE INDEX IF NOT EXISTS idx_obs_symbol ON signal_observations(symbol, timestamp);
CREATE INDEX IF NOT EXISTS idx_labels_obs ON signal_labels(observation_id);
```

■ **BAJO-3:** *SCAN_INTERVAL_SECONDS=30 fijo — debería ser adaptativo al régimen*

En HIGH_VOLATILITY, 30s es eterno. En CHOPPY_MARKET, 30s puede ser demasiado frecuente (desperdiciar CPU). Un intervalo adaptativo de 10s en volatilidad alta y 60s en mercado choppy mejoraría latencia en momentos críticos.

■ **BAJO-4:** *El _risk_adjusted_score del allocator no considera R:R*

Archivo: app/portfolio_allocator.py:103–108

```
@staticmethod
def _risk_adjusted_score(signal: Signal) -> float:
    rr_bonus = min(signal.risk_reward_ratio, 3.0) * 2.0
    warning_penalty = len(signal.warnings) * 4.0
    liquidity_bonus = 3.0 if signal.symbol in {"BTCUSDT", "ETHUSDT", "SOLUSDT"} else 0.0
    return signal.confidence_score + rr_bonus + liquidity_bonus - warning_penalty
```

El rr_bonus usa el R:R calculado que incluye el error de CRÍTICO-1. Cuando se corrija el R:R real, este ranking mejorará automáticamente. El peso del R:R (×2) es bajo — debería ser más prominente en el ranking.

■ **BAJO-5:** *Timeout de 10s en requests puede ser demasiado bajo en Railway*

Archivo: app/bitget_client.py:130 (timeout=10)

En servidores con latencia variable (Railway en Europa → Bitget en Asia), 10s puede ser insuficiente en momentos de carga. Un timeout de 15–20s con retry reducido (2 intentos) tiene mejor balance.

■ **BAJO-6: Falta de normalización de slippage por liquidez del símbolo**

El slippage está hardcoded en 0.03% para todos los símbolos. DOGEUSDT y DOTUSDT tienen peor liquidez que BTCUSDT. El slippage real en alts puede ser 0.1–0.3%, especialmente en posiciones >\$100 notional.

VII. ANÁLISIS DE EDGE ESPERADO — ¿EL BOT GANA DINERO?

Matemáticas del sistema actual (hipotéticas con datos disponibles)

Parámetros base:

- Balance: \$40 USDT
- Margen por trade: \$12 USDT
- Leverage: 3x → Notional: \$36
- Stop mínimo: 0.6% → Risk: $\$36 \times 0.6\% = \0.216
- TP1 (1.6x): $\$36 \times 0.96\% = \0.346
- Fees round-trip: $\$36 \times 0.12\% = \0.043
- Slippage: $\$36 \times 0.03\% = \0.011

Ganancia neta TP1: $\$0.346 - \$0.043 - \$0.011 = \0.292

Pérdida neta SL: $\$0.216 + \$0.043 + \$0.011 = \0.270

R:R neto real: $0.292 / 0.270 = 1.08$ (vs. 1.6 declarado)

Break-even win rate necesario: $1 / (1 + 1.08) = 48\%$

Conclusión crítica: Si el bot tiene win rate < 48%, PIERDE dinero. Para un sistema de trading algorítmico en futuros con comisiones taker, **el break-even real es ~48%, no ~38% como el R:R de 1.6 sugeriría**. El sistema necesita ser MUY preciso en sus señales.

Con las correcciones propuestas

Con TPs dinámicos (TP1 = 2.0x en tendencia):

- R:R neto: $(2.0 \times 0.6 - 0.15\%) / (0.6 + 0.15\%) = 1.05 / 0.75 = 1.40$
 - Break-even win rate: $1 / (1 + 1.40) = 42\%$ → mucho más alcanzable
-

VIII. OPORTUNIDADES DE MEJORA PARA GANAR MÁS DINERO

Prioridad ALTA (implementar primero):

1. **Activar trailing stop real** — Mover SL a break-even en TP1. Esto transforma trades mediocres en trades neutros (sin pérdida), mejorando el Expectancy directamente.
2. **Corregir R:R neto (CRÍTICO-1)** — El más impactante. Asegura que el sistema opera solo cuando el edge matemático real es positivo.

3. **TP dinámico por régimen** (CRÍTICO-2) — En TREND_UP operar con TP más lejano aumenta el tamaño de los ganadores sin cambiar el win rate.
4. **Paralelizar fetch de mercado** (ALTO-1) — Permite bajar SCAN_INTERVAL_SECONDS a 15s sin aumentar latencia. Más señales capturadas.
5. **WebSocket para monitor de posiciones** (ALTO-2) — Cierre de posiciones en tiempo real, no con 5-30s de lag.

Prioridad MEDIA (segunda fase):

6. **Activar MetaModel en modo filter** — Una vez con 300+ labels, puede mejorar el win rate eliminando los trades de peor calidad dentro de los que ya superan el score mínimo.
7. **Calibrar score mínimo con datos reales** — Usar los datos MFE/MAE acumulados para encontrar el score umbral donde el win rate sea consistentemente >52%.
8. **Implementar circuit breaker por drawdown %** (CRÍTICO-5) — Protección más inteligente que no penaliza streaks de pequeñas pérdidas normales.
9. **Funding rate como coste explícito** (MEDIO-7) — Evitar quedar atrapado pagando funding mientras esperamos TP2.

VIII-B. HALLAZGOS ADICIONALES DE SEGURIDAD Y ARQUITECTURA

■ CRÍTICO-6: Cierre de emergencia sin reintento — posición puede quedar sin stop

Archivo: app/execution_engine.py (bloque de stop loss fallido)

```
stop_ok = self._place_stop(...)
if not stop_ok:
    close = self.client.close_position_market(...) # Si ESTO falla → posición real sin protección
```

Problema: Si la API falla al colocar el stop loss Y también falla el cierre de emergencia, la posición queda abierta en el exchange real **sin ninguna protección**. No hay reintento, no hay alerta Telegram de emergencia en ese path específico.

Corrección:

```
for attempt in range(3):
    try:
        close = self.client.close_position_market(...)
        break
    except Exception as exc:
        if attempt == 2:
            self.telegram.critical(f"CRÍTICO: Cierre de emergencia falló 3 veces en {symbol}: {exc}")
")
```

■ CRÍTICO-7: Falta de idempotencia — inconsistencia exchange vs DB si el proceso muere

Problema: Si el proceso se interrumpe entre ejecutar la orden en el exchange (éxito) y registrarla en la base de datos (no ejecutado), el bot no sabe de la posición al reiniciar. En el próximo ciclo intentará abrir otra posición en el mismo símbolo, ignorando la ya abierta.

El campo `clientoid` ya se genera por trade, pero no se usa para hacer reconciliación de estado al startup.

Corrección: Guardar estado `PENDING_EXECUTION` en DB ANTES de enviar la orden, y usar `clientOid` para reconciliar en el startup.

■ ALTO-9: Riesgo de SQL Injection en ALTER TABLE dinámico

Archivo: `app/database.py:710-730`

```
# PostgreSQL:
self._execute(conn, f"ALTER TABLE signal_observations ADD COLUMN IF NOT EXISTS {name} {spec}")
# SQLite:
conn.execute(f"ALTER TABLE signal_observations ADD COLUMN {name} {spec}")
```

Estado actual: Los valores de `name` y `spec` son hardcodedados en un diccionario interno → actualmente SEGURO. Sin embargo, si en el futuro se permite input externo para nombres de columnas (ej. features dinámicas desde API), esto es una vulnerabilidad de inyección SQL.

Corrección preventiva: Whitelist de nombres permitidos:

```
ALLOWED_COLUMN_NAMES = {"feature_x", "feature_y", ...}
assert name in ALLOWED_COLUMN_NAMES, f"Nombre de columna no permitido: {name}"
```

■ MEDIO-10: Validación insuficiente de parámetros de riesgo — valores peligrosos permitidos

Archivo: `app/config.py:410`

```
max_risk_per_trade=env_float(os.getenv("MAX_RISK_PER_TRADE"), 0.025),
# Sin validación de rango – alguien puede poner MAX_RISK_PER_TRADE=0.99
```

No hay validación que impida configurar `MAX_RISK_PER_TRADE=0.99` (99% del balance por trade) o `DEFAULT_LEVERAGE=100` (aunque el `max_leverage` cap lo limita a 5, el parámetro mismo no tiene validación cruzada completa).

IX. RIESGOS ESPECÍFICOS ANTES DE ACTIVAR LIVE

Riesgo | Probabilidad | Impacto | Mitigación

--- | --- | --- | ---

R:R declarado vs. real | Alto | Crítico | Corregir CRÍTICO-1

Posición sobredimensionada por balance obsoleto | Medio | Alto | Corregir CRÍTICO-4

Stop falso por whipsaw ATR | Alto | Medio | Corregir CRÍTICO-3

Reloj host desincronizado | Medio | Crítico | Añadir check de drift

Trailing stop no ejecutado | Alto | Medio | Implementar en PositionManager

Fetch secuencial — señales desfasadas | Alto | Medio | Paralelizar

X. PLAN DE ACCIÓN RECOMENDADO (PRIORIZADO)

Sprint 1 — Correcciones de bugs críticos (antes de cualquier live)

1. [] Corregir cálculo R:R neto con fees y slippage en `signal_engine.py:147`
2. [] Corregir lógica de stop loss en `signal_engine.py:234-241`

3. [] Mover balance refresh ANTES de `risk_manager.validate_signal` en `main.py:481`
4. [] Fix dead code en `risk_manager._max_positions_for_balance:552`
5. [] Añadir check de clock drift al startup

Sprint 2 — Mejoras de edge y rendimiento

6. [] Implementar TP dinámico por régimen en `signal_engine.py`
7. [] Implementar trailing stop real en `position_manager.py` / `paper_trader.py`
8. [] Paralelizar `market_data.fetch_all()` con `ThreadPoolExecutor`
9. [] Añadir funding rate como coste en cálculo de R:R

Sprint 3 — Mejoras de sistema y ML

10. [] Añadir índices DB para queries calientes
11. [] Cache incremental de indicadores técnicos
12. [] Activar `MetaModel filter` cuando sample size suficiente
13. [] Intervalo de scan adaptativo por régimen
14. [] Calibrar `MIN_SCORE_TO_TRADE` con datos MFE/MAE reales

XI. CONCLUSIÓN

El bot está bien diseñado conceptualmente. La arquitectura de research (MFE/MAE, labeling, meta-model) es sofisticada y diferencia este sistema de la mayoría. La gestión de riesgo tiene múltiples capas correctamente implementadas.

Sin embargo, los bugs CRÍTICOS-1 y CRÍTICO-2 juntos crean un sistema que opera con un R:R declarado de 1.6 pero real de ~1.08, lo cual requiere un win rate de 48%+ para ser rentable. En mercados eficientes con comisiones taker, ese win rate es exigente.

Antes de activar live trading, son obligatorias las correcciones de CRÍTICO-1, CRÍTICO-3 y CRÍTICO-4. Las demás pueden implementarse progresivamente.

El sistema tiene todos los ingredientes para ser rentable: buena arquitectura de datos, infraestructura de investigación, y lógica de señales razonable. Las correcciones propuestas pueden elevar el Expectancy estimado de ~0 a +0.15–0.25 por trade, lo cual compuesto en docenas de trades mensuales representa una diferencia sustancial en el PnL final.

Auditoría realizada sobre el estado del código en rama `claude/audit-trading-bot-64DAW`. Fecha: 2026-05-19.